

Optimization Algorithms



CS4234

Week 3

Maximum Satisfiability

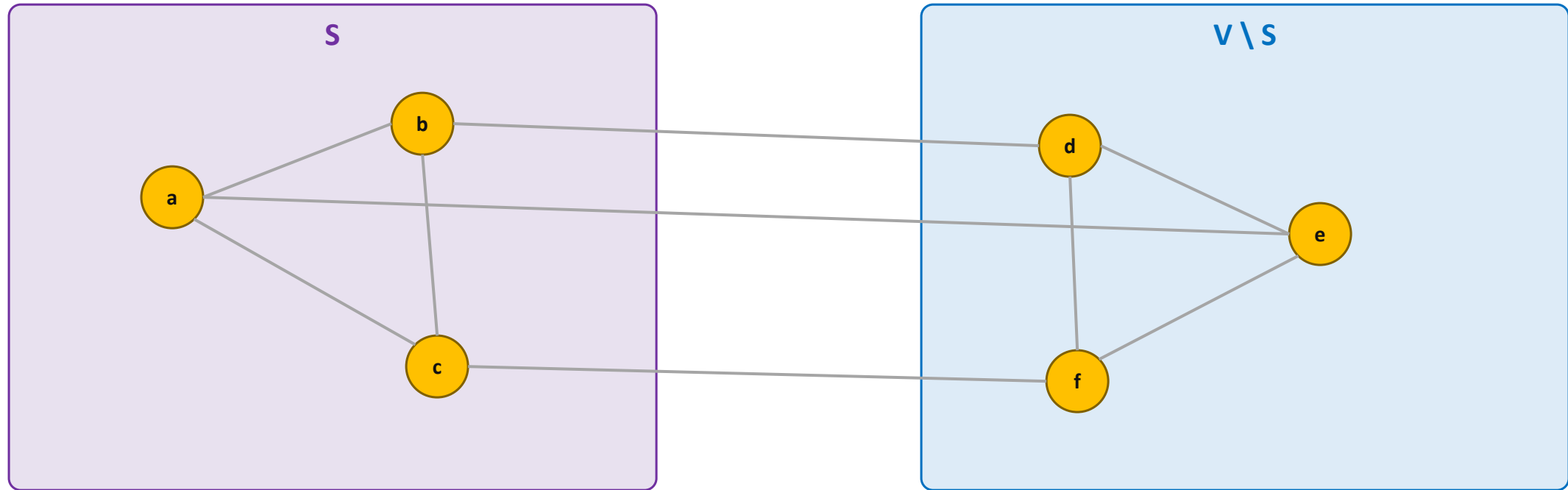
Diptarka Chakraborty

Announcement

- Mini-project details are available in Canvas.
- Work in groups of 3–4 and read the instruction document.
- Understand the problem, compare one or two solution ideas, and implement when feasible.
- You may use an LLM to formalize your ideas to see if it provides any improvement for the problem, in which subcase it improves, or to implement it.
- Use Google Scholar to find useful baselines and heuristics.

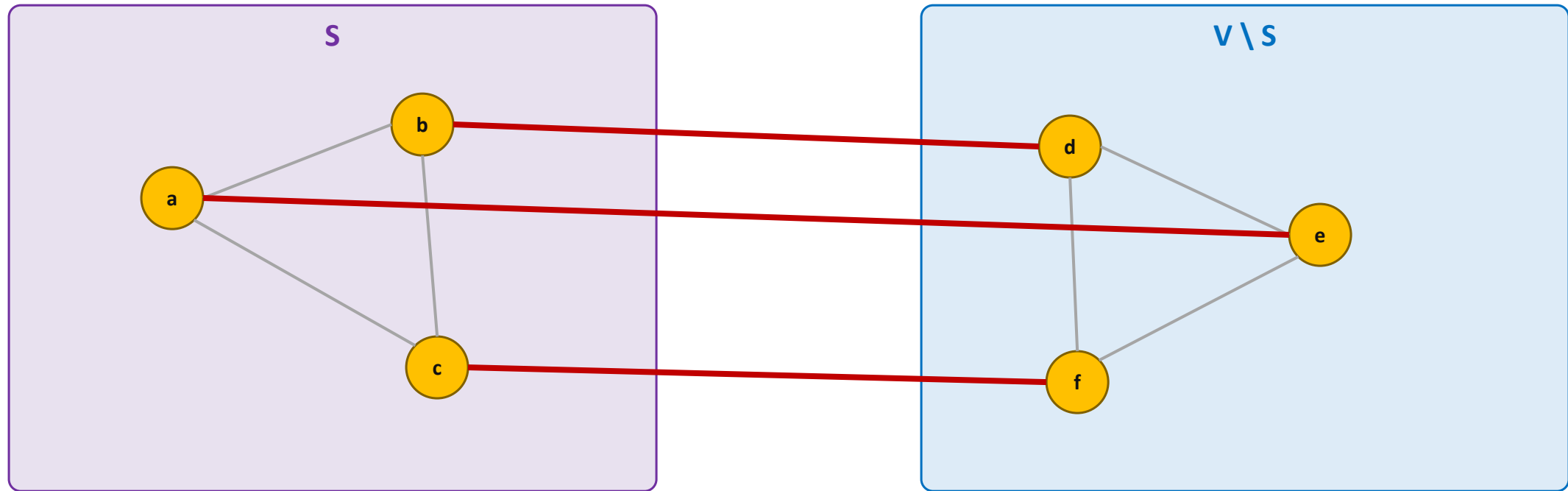
For the mini-project: understand → compare → test.

Max-Cut: separate vertices to maximize crossing weight



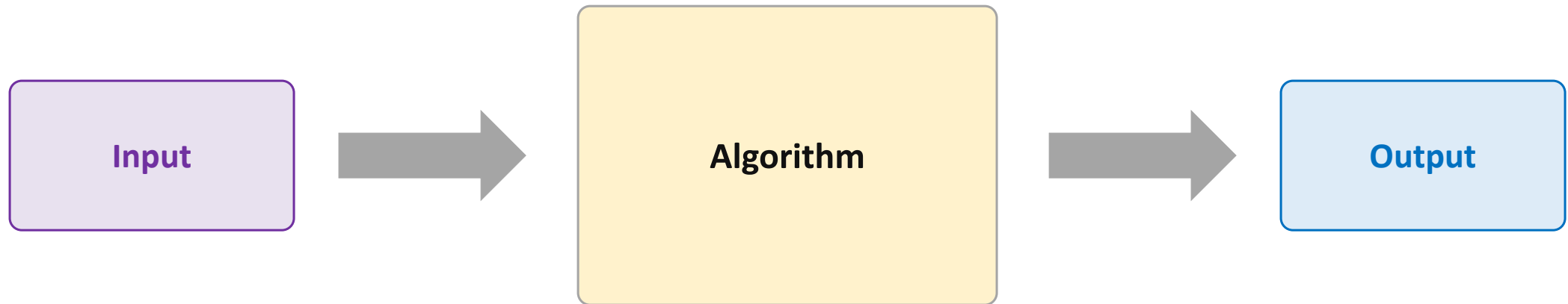
A cut partitions the vertices; its value is the total weight crossing the partition.

Max-Cut is a canonical randomized-approximation problem



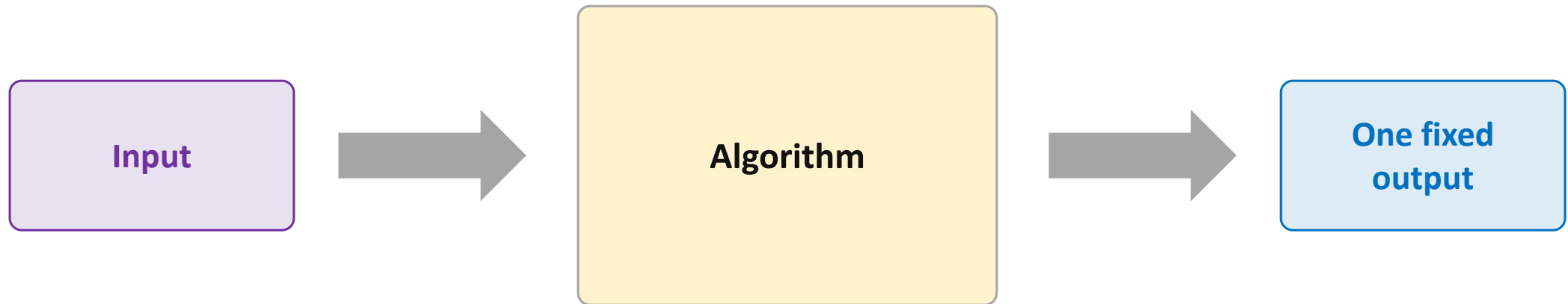
Only edges with exactly one endpoint in S contribute to the cut.

An algorithm maps each input to an output



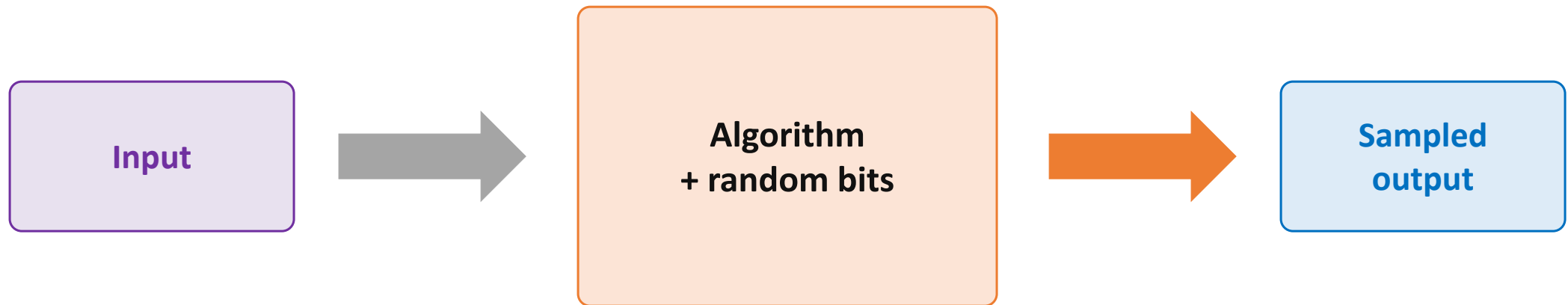
The output and running time are functions of the input.

A deterministic algorithm repeats the same behavior



Fix the input: both the output and execution path are fixed.

A randomized algorithm samples an output



Fix the input: random bits still create a distribution over outputs.

Measure randomized approximation in expectation

Minimization

$$E[\text{cost}(A)] / \text{OPT} \leq \rho(n)$$

Maximization

$$\text{OPT} / E[\text{cost}(A)] \leq \rho(n)$$

The expectation is over the algorithm's internal random choices.

Expectation is one guarantee—not the only one

In expectation

Analyze $E[\text{cost}(A)]$.

Often derandomizable by the method of conditional expectation.

With high probability

Bound the failure probability.

Tools: concentration, repetition, success amplification.

Choose the guarantee that matches how the algorithm will be used.

Expectation is a weighted average of outcomes



For a discrete random variable X :

$$E[X] = \sum_x x \cdot \Pr[X = x]$$

For a fair die: $E[X] = (1+2+3+4+5+6)/6 = 3.5$

Expectation summarizes a distribution—weighted average of attainable outcomes.

Linearity lets us analyze pieces independently

For any random variables X and Y and constant a :

$$E[X + Y] = E[X] + E[Y] \quad \text{and} \quad E[aX] = aE[X]$$

No independence assumption is required.

Express the objective as a sum of indicators, then analyze one indicator at a time.

Random cut: place every vertex independently

- 1 Start with $S = \emptyset$.
- 2 For each vertex v , put v in S with probability $1/2$.
- 3 Return the cut $(S, V \setminus S)$.

**One random bit
per vertex**

Independence makes each edge easy to analyze.

A random cut keeps half the total edge weight

Let X_e indicate whether edge e crosses the cut.

$$E[w(\text{cut})] = \sum_e w(e)E[X_e] = \frac{1}{2}\sum_e w(e)$$

The remaining question is why $\Pr[X_e = 1] = 1/2$.

The expected cut value is exactly half of the graph's total weight.

Each edge crosses with probability $1/2$

For $e = (u,v)$, the endpoints receive independent fair bits.

$$\Pr[X_e = 1] = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

Therefore $E[X_e] = 1/2$, and linearity sums the contribution of every edge.

Analyze one edge; linearity does the rest.

Random cut is a 2-approximation

$$E[w(\text{cut})] \geq \frac{1}{2} \cdot \text{OPT}$$

OPT is at most the total edge weight, while the random cut gets half of that total weight in expectation.

A simple local random choice yields a global factor-1/2 guarantee.

Max-SAT asks us to satisfy as many clauses of a CNF as possible

Literal

A Boolean variable x_j or its negation $\neg x_j$.

Clause

An OR of literals, e.g. $C_i = x_1 \vee \neg x_2 \vee x_4$.

CNF

An AND of clauses: $\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$.

Max-SAT

Choose an assignment maximizing the satisfied clause weight.

SAT asks whether all clauses can be satisfied; Max-SAT optimizes how many.

A concrete formula separates satisfying assignments

Formula

$$\Phi = (\neg x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$$

Two assignments

Satisfying: $x_1=T, x_2=F, x_3=F, x_4=T$

Not satisfying: $x_1=T, x_2=F, x_3=F, x_4=F$

The objective counts the clauses made true by the assignment.

Quick check: random assignment for Max-E2SAT

Every clause has exactly two literals. If every variable is independently true with probability $1/2$, what approximation factor follows?

A 4

B 2

C $4/3$

D 1

Compute the probability that one two-literal clause is satisfied.

A two-literal clause is satisfied with probability $3/4$

The clause fails only when both literals are false.

$$\Pr[\text{clause satisfied}] = 1 - \left(\frac{1}{2}\right)^2 = \frac{3}{4}$$

So $E[\# \text{ satisfied}] = \frac{3}{4}m \geq \frac{3}{4} \cdot \text{OPT}$ because $\text{OPT} \leq m$.

Random assignment is a $4/3$ -approximation for Max-E2SAT.

Weighted Max-SAT uses clause values

Weight

Each clause C_i has nonnegative weight w_i .

Indicator

$\tau(C_i) = 1$ if the assignment satisfies C_i ; otherwise 0.

Objective

$\text{cost}(\tau) = \sum_i w_i \tau(C_i)$.

Goal

Choose τ maximizing $\text{cost}(\tau)$.

Linearity of expectation also works for weighted clauses.

Algo1: flip an unbiased coin for every variable

- 1 For each variable x_j , independently set $x_j = \text{true}$ with probability $1/2$.
- 2 Evaluate which clauses are satisfied.
- 3 Return the sampled assignment τ .

Goal:
bound $E[\text{cost}(\tau)]$

Start with the simplest possible randomized assignment.

Longer clauses are easier to satisfy

If clause C_i has ℓ_i literals, it fails only when every literal is false.

$$\Pr[C_i \text{ satisfied}] = 1 - 2^{-\ell_i}$$

The worst case is $\ell_i = 1$, where the probability is only $1/2$.

Clause length controls the success probability of unbiased rounding.

Algo1 guarantees half of optimum

$$E[\text{cost}(\tau)] = \sum_i w_i(1 - 2^{-\ell_i}) \geq \frac{1}{2} \sum_i w_i \geq \frac{1}{2} \cdot \text{OPT}$$

Every clause contributes at least half its weight in expectation.

Unbiased coins give a 2-approximation for Max-SAT.

Can biased coins beat factor 1/2?

Unbiased

$$\Pr[x_j = \text{true}] = 1/2$$

Symmetric, but singleton clauses remain a bottleneck.

Biased

$$\Pr[x_j = \text{true}] = p$$

Tune p to favor the literal structure after preprocessing.

The next algorithm trades symmetry for a stronger worst-case bound.

Algo2: set each variable true with probability p

- 1 Choose a bias $p \in [0,1]$.
- 2 Independently set $x_j = \text{true}$ with probability p and false with probability $1-p$.
- 3 Return the sampled assignment.

**Choose p
to maximize the worst clause
probability**

The analysis reveals which p balances the competing clause types.

First assume every clause has at least two literals

Let a_i and b_i be the positive and negative literal counts in C_i , with $a_i + b_i = \ell_i \geq 2$.

$\Pr[C_i \text{ satisfied}] = ?$

Assume $p \geq 1/2$.

With length at least two, every clause succeeds with probability at least $1 - p^2$.

First assume every clause has at least two literals

Let a_i and b_i be the positive and negative literal counts in C_i , with $a_i + b_i = \ell_i \geq 2$.

$$\Pr[C_i \text{ satisfied}] = 1 - p^{b_i}(1 - p)^{a_i} \geq 1 - p^{\ell_i}$$

Assume $p \geq 1/2$.

With length at least two, every clause succeeds with probability at least $1 - p^2$.

Under the length assumption, Algo2 earns $1-p^2$

$$E[\text{cost}(\tau)] \geq (1-p^2) \sum_i w_i \geq (1-p^2) \cdot \text{OPT}$$

This bound improves as p moves away from 1—until singleton clauses intervene.

For clauses of length at least two, the guarantee is $1-p^2$.

Positive singleton clauses earn probability p

A clause containing only x_j is satisfied exactly when $x_j = \text{true}$.

$$\Pr[x_j \text{ satisfied}] = p$$

We must balance p for positive singletons against $1-p^2$ for longer clauses.

The worst clause type determines the global approximation factor.

Balance the two clause guarantees

$$\text{guarantee}(p) = \min\{ p, 1-p^2 \} \geq 0.618$$

Set $p = 1-p^2$ so neither clause type is the bottleneck ($p = (\sqrt{5} - 1)/2$).

The optimum bias is where the two lower bounds meet.

Preprocess conflicting singleton clauses

(From now assume without loss of generality)

1

Each single literal clause appear at most once; otherwise increase its weight

2

Among single literal clauses: $\text{weight of clause } x_i \geq \text{weight of clause } \bar{x}_i$; otherwise, consider new variable $y_i = \bar{x}_i$ and replace all the occurrences

3

Similarly, both x_i and $\bar{x}_i$ must occur (if $\bar{x}_i$ occurs); otherwise, consider new variable $y_i = \bar{x}_i$ and replace all the occurrences

Preprocessing creates the structure needed for a useful bias.

At most N singleton weight is unavoidable

Let N be the total *weight* of $\bar{x}_j$ appearing as a clause

$$\text{OPT} \leq \sum_i w_i - N$$

For each pair x_j and $\neg x_j$, every assignment must lose at least the smaller weight.

The preprocessing loss is also a certified upper-bound correction.

Analyze the remaining formula with positive singletons

Let U contain every clause except the removed negative singleton clauses.

$$\mathbb{E}[\text{cost on } U] \geq \min\{p, 1-p^2\} \cdot \sum_{C_i \in U} w_i$$

The same two cases now cover every surviving clause.

After preprocessing, one bias parameter controls the whole formula.

Combine the preprocessing and random-assignment bounds

$$\mathbf{E}[\text{cost}(\tau)] \geq \min\{p, (1 - p^2)\} (\sum_i w_i - N) \geq \min\{p, (1 - p^2)\} OPT$$

The structural upper bound absorbs the weight removed during preprocessing.

A lower bound on the algorithm meets an upper bound on OPT.

The golden-ratio bias gives 0.618

Solve $p = 1 - p^2$ in $[0,1]$.

$$p = (\sqrt{5} - 1)/2 \approx 0.618$$

At this value, $p = 1 - p^2$, so the two worst-case clause bounds are equal.

Algo2 achieves a 1.619-approximation for Max-SAT.

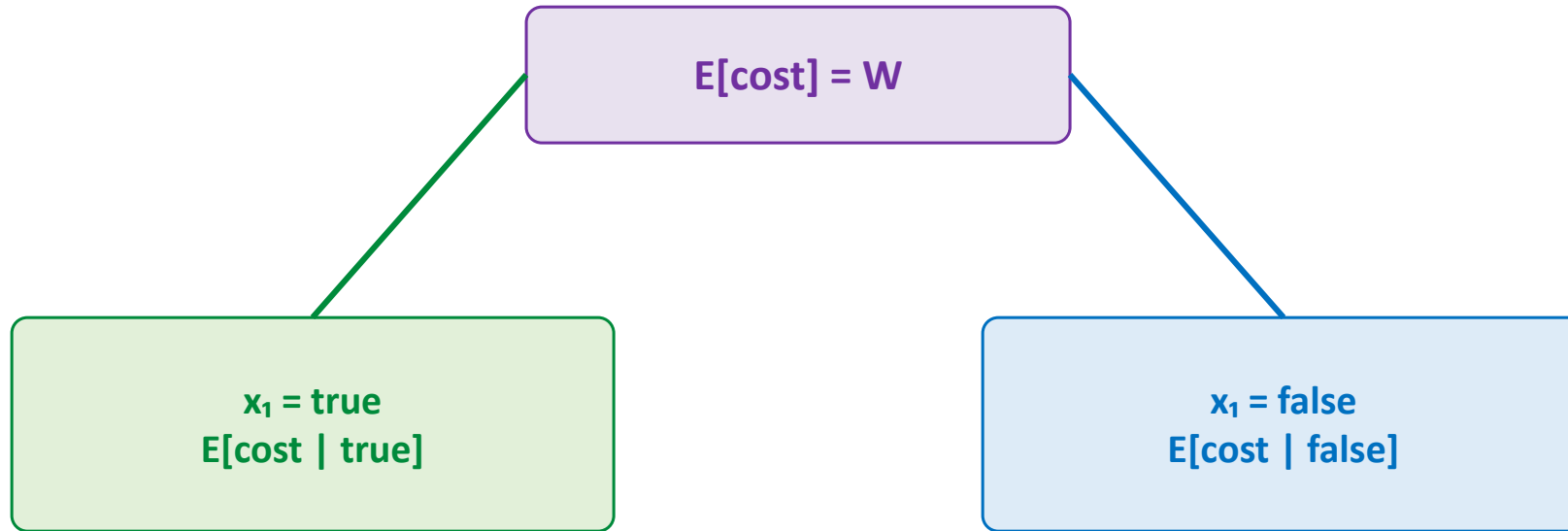
Biased coins improve the 2-approximation guarantee

$$\mathbb{E}[\text{cost}(\text{Algo2})] \geq 0.618 \cdot \text{OPT}$$

Next questions: Can we remove the randomness? Can LP relaxation improve the factor further?

The proof combines preprocessing, bias optimization, and expectation.

Derandomization follows the better conditional branch



Choose the branch whose conditional expectation is at least the current expectation.

Random choices become deterministic decisions without decreasing the guarantee.

Conditional expectation turns choices into decisions

Suppose the current partial assignment has conditional expectation W and $p = \Pr[x_j = \text{true}]$

$$W = p \cdot E[\text{cost} \mid x_j = \text{true}] + (1-p) \cdot E[\text{cost} \mid x_j = \text{false}]$$

At least one branch has conditional expectation at least W ; choose it deterministically.

A weighted average cannot exceed both of its components.

The first variable splits the original expectation

$$E[\text{cost}(\tau)] = \frac{1}{2}E[\text{cost} \mid x_1=\text{true}] + \frac{1}{2}E[\text{cost} \mid x_1=\text{false}]$$

Therefore $\max\{E[\text{cost} \mid \text{true}], E[\text{cost} \mid \text{false}]\} \geq E[\text{cost}(\tau)]$.

One of the two first decisions preserves the expected guarantee.

Choose the branch with larger conditional expectation

Set $x_1 = \text{true}$

Compute $E[\text{cost} \mid x_1=\text{true}]$

Set $x_1 = \text{false}$

Compute $E[\text{cost} \mid x_1=\text{false}]$

Commit to the better branch, then continue with the next variable.

Repeat until every variable is fixed

1

Compute both conditional expectations
for x_j .

2

Choose the larger one and fix x_j .

3

Continue with x_{j+1} until the assignment
is complete.

The conditional expectation never decreases along the chosen path.

The final deterministic assignment keeps the guarantee

After the last variable is fixed, no randomness remains.

$$\text{cost}(\tau\text{-det}) \geq E[\text{cost}(\tau\text{-random})]$$

If the randomized algorithm was an α -approximation in expectation, the deterministic assignment is also an α -approximation.

Derandomization converts an existential expectation into a concrete solution.

Derandomization is efficient when expectations are computable

Needed

Compute the two conditional expectations at each variable.

Cost

A polynomial number of variables times polynomial work per decision.

Max-SAT

Clause contributions update locally, so the computation is efficient.

The method works when conditional expectations are polynomial-time computable.

Randomization gives two reusable techniques

Analysis

Use indicators and linearity of expectation.

Design

Tune a biased distribution to balance worst cases.

Derandomize

Follow the branch with the larger conditional expectation.

Exercise: derandomize the random Max-Cut algorithm by conditional expectation.

References

- **Max-SAT:**

- Chapter 5 of [Williamson and Shmoys]

- Chapter 16 of [Vazirani]